Experiment No: 7

TITLE: TO CREATE THE FOLLOWING DATABASE SCHEMA AND RUN SQL QUERIES TO RETRIEVE THE INFORMATION AS INSTRUCTED.

1. Objectives

1.1 To design and create an employee table in MySQL.

1.2 To apply SQL queries for data retrieval using aliasing, sorting, and conditions.

1.3 To understand filtering, range-based conditions, and computed columns in SQL.

2. THEORY

Structured Query Language (SQL) is used to manage and query relational databases. For this experiment, we design an employee table to store details such as ID, name, post, salary, date of birth, gender, address, and contact. SQL provides powerful features such as **aliases**, which help rename columns or tables for better readability, and **conditional operators** like BETWEEN, NOT BETWEEN, and ORDER BY for filtering and sorting results.

SQL Queries

Here are the SQL queries to retrieve the requested information based on the employee table schema.

- **Name, post, and salary of the employee using column alias and table alias.**
  - **Query:** SELECT e.name AS employee_name, e.post AS job_title, e.salary AS monthly_salary FROM employee AS e;
  - **Explanation:** This query selects the name, post, and salary columns from the employee table. It uses e as a table alias for employee to shorten the query. It also uses column aliases (employee_name, job_title, monthly_salary) to provide more descriptive names for the output columns.
- **List of different posts.**
  - **Query:** SELECT DISTINCT post FROM employee;
  - **Explanation:** The DISTINCT keyword is used to retrieve a unique list of values from the post column, avoiding duplicates.
- **List out name, salary, and tax amount (15% of salary).**
  - **Query:** SELECT name, salary, salary * 0.15 AS tax_amount FROM employee;
  - **Explanation:** This query calculates a new column on the fly by multiplying the salary by 0.15 (15%). The result of this calculation is given an alias, tax_amount, to make the column header more readable.
- **Name of all the employees in ascending order.**
  - **Query:** SELECT name FROM employee ORDER BY name ASC;
  - **Explanation:** The ORDER BY clause sorts the result set by the name column. The ASC keyword specifies an ascending order (A to Z), which is the default, so it can also be omitted.
- **Name of the employee having salary between 40,000 and 50,000.**

- **Query:** SELECT name FROM employee WHERE salary BETWEEN 40000 AND 50000;
- **Explanation:** The WHERE clause filters the rows. The BETWEEN operator selects all rows where the salary value is within the specified inclusive range of 40,000 and 50,000.

- **Name of the employee having salary not between 40,000 and 50,000.**
  - **Query:** SELECT name FROM employee WHERE salary NOT BETWEEN 40000 AND 50000;
  - **Explanation:** The NOT BETWEEN operator is the inverse of BETWEEN. It filters for all rows where the salary value falls outside of the inclusive range of 40,000 and 50,000.

3. DEMOSTRATION

Employee table:

```
MariaDB [ashishlabreport]> desc employee;
+---------+---------+------+-----+---------+-------+
| Field   | Type    | Null | Key | Default | Extra |
+---------+---------+------+-----+---------+-------+
| eid     | int(11) | NO   | PRI | NULL    |       |
| name    | text    | YES  |     | NULL    |       |
| post    | text    | YES  |     | NULL    |       |
| salary  | int(11) | YES  |     | NULL    |       |
| dob     | text    | YES  |     | NULL    |       |
| gender  | text    | YES  |     | NULL    |       |
| address | text    | YES  |     | NULL    |       |
| contact | text    | YES  |     | NULL    |       |
+---------+---------+------+-----+---------+-------+
8 rows in set (0.025 sec)
```

Inserted Data:

```
MariaDB [ashishlabreport]> select * from employee;
+-----+----------------+------------+--------+------------+--------+------------+------------+
| eid | name           | post       | salary | dob        | gender | address    | contact    |
+-----+----------------+------------+--------+------------+--------+------------+------------+
| 101 | rajesh sharma  | manager    |  75000 | 1985-04-12 | male   | kathmandu  | 9841001234 |
| 102 | sita basnet    | developer  |  45000 | 1992-08-25 | female | pokhara    | 9801234567 |
| 103 | narayan karki  | accountant |  55000 | 1988-11-03 | male   | biratnagar | 9851122334 |
| 104 | isha thapa     | developer  |  48000 | 1995-02-14 | female | lalitpur   | 9865432109 |
| 105 | sanjay rai     | team lead  |  65000 | 1989-06-30 | male   | bhaktapur  | 9815012345 |
| 106 | pratima dhakal | hr manager |  60000 | 1987-10-10 | female | chitwan    | 9803456789 |
| 107 | bikram tamang  | developer  |  42000 | 1993-01-20 | male   | dharan     | 9823456789 |
| 108 | pujan magar    | accountant |  52000 | 1990-09-05 | male   | birtamod   | 9845678901 |
| 109 | aayusha ghimire | developer |  40000 | 1994-03-08 | female | kathmandu  | 9801234560 |
| 110 | sushil gurung  | hr manager |  62000 | 1986-07-22 | male   | pokhara    | 9865432101 |
+-----+----------------+------------+--------+------------+--------+------------+------------+
10 rows in set (0.001 sec)
```

SQL queries to retrieve the following information.

1. **Name, post and salary of the employee using column alias and table alias.**

```
MariaDB [ashishlabreport]> select name as employee_name, post as employee_post,
 salary as monthly_salary from employee as e;
+-----------------+---------------+----------------+
| employee_name   | employee_post | monthly_salary |
+-----------------+---------------+----------------+
| rajesh sharma   | manager       |          75000 |
| sita basnet     | developer     |          45000 |
| narayan karki   | accountant    |          55000 |
| isha thapa      | developer     |          48000 |
| sanjay rai      | team lead     |          65000 |
| pratima dhakal  | hr manager    |          60000 |
| bikram tamang   | developer     |          42000 |
| pujan magar     | accountant    |          52000 |
| aayusha ghimire | developer     |          40000 |
| sushil gurung   | hr manager    |          62000 |
+-----------------+---------------+----------------+
10 rows in set (0.001 sec)
```

2. **List of different posts.**

```
MariaDB [ashishlabreport]> select distinct post from employee;
+------------+
| post       |
+------------+
| manager    |
| developer  |
| accountant |
| team lead  |
| hr manager |
+------------+
5 rows in set (0.022 sec)
```

3. **List out name, salary and tax amount (15% of salary).**

```
MariaDB [ashishlabreport]> select name, salary, salary * 0.15 as tax_amount fro
m employee;
+-----------------+--------+------------+
| name            | salary | tax_amount |
+-----------------+--------+------------+
| rajesh sharma   |  75000 |   11250.00 |
| sita basnet     |  45000 |    6750.00 |
| narayan karki   |  55000 |    8250.00 |
| isha thapa      |  48000 |    7200.00 |
| sanjay rai      |  65000 |    9750.00 |
| pratima dhakal  |  60000 |    9000.00 |
| bikram tamang   |  42000 |    6300.00 |
| pujan magar     |  52000 |    7800.00 |
| aayusha ghimire |  40000 |    6000.00 |
| sushil gurung   |  62000 |    9300.00 |
+-----------------+--------+------------+
10 rows in set (0.001 sec)
```

4. **Name of all the employees in ascending order.**

```
MariaDB [ashishlabreport]> select name from employee order by name asc;
+-----------------+
| name            |
+-----------------+
| aayusha ghimire |
| bikram tamang   |
| isha thapa      |
| narayan karki   |
| pratima dhakal  |
| pujan magar     |
| rajesh sharma   |
| sanjay rai      |
| sita basnet     |
| sushil gurung   |
+-----------------+
10 rows in set (0.001 sec)
```

5. **Name of the employee having salary between 40,000 and 50,000.**

```
MariaDB [ashishlabreport]> select name from employee where salary between 40000
 and 50000;
+-----------------+
| name            |
+-----------------+
| sita basnet     |
| isha thapa      |
| bikram tamang   |
| aayusha ghimire |
+-----------------+
4 rows in set (0.001 sec)
```

6. **Name of the employee having salary not between 40,000 and 50,000.**

```
MariaDB [ashishlabreport]> select name from employee where salary not between 4
0000 and 50000;
+----------------+
| name           |
+----------------+
| rajesh sharma  |
| narayan karki  |
| sanjay rai     |
| pratima dhakal |
| pujan magar    |
| sushil gurung  |
+----------------+
6 rows in set (0.001 sec)
```

4. RESULT AND DISCUSSION

The queries were executed on the employee table successfully. Using table and column aliases, employee details such as name, post, and salary were displayed with more descriptive labels. The distinct keyword listed different posts without repetition. The computation salary*0.15 correctly displayed the tax amount for each employee. Sorting with order by arranged the employees' names in ascending order. The use of between and not between conditions helped filter employees based on their salary range. This demonstrated the practical use of SQL clauses for efficient data retrieval.

5. CONCLUSION

This experiment successfully demonstrated the creation of an employee database and execution of various SQL queries. We learned to use aliases for readability, distinct values for uniqueness, computed columns for calculations, sorting for ordered results, and conditional queries for filtering based on ranges. Overall, this enhanced understanding of SQL's querying power in handling real-world database operations.

Experiment No: 8

TITLE: TO CREATE EMPLOYEE DATABASE SCHEMA AND RETRIEVE INFORMATION USING SQL AGGREGATE QUERIES

1. OBJECTIVES

1.2 To implement aggregate functions like count(), avg(), and sum().

1.3 To use subqueries for retrieving maximum and comparison-based data.

1.4 To analyze salary distribution among employees using SQL.

2. THEORY

In SQL, **aggregate functions** are used to perform calculations on sets of values and return a single summarizing value. Functions such as COUNT(), AVG(), and SUM() are essential for analyzing data in a database. These functions help determine the number of records, average values, or total amounts. SQL also supports the use of **GROUP BY** to organize records into groups, and **subqueries** to retrieve values for comparison against other queries.

A key companion to aggregate functions is the GROUP BY clause. This clause is used to group rows that have the same values in specified columns into summary rows. For example, by using GROUP BY, we can calculate the average salary for each distinct job post or for each gender, rather than just for the entire table.

Additionally, we will demonstrate the use of a **subquery**. A subquery is a query nested inside another query. The inner query, or subquery, executes first, and its result is used by the outer query. This is a powerful technique that allows for more complex data retrieval, such as finding the name of an employee who has the single highest salary in the entire table.

SQL Queries

Here are the SQL queries to retrieve the requested information using aggregate functions based on the employee table schema.

- **Count the total number of employees in the 'employee' table.**
  - **Query:** SELECT COUNT(*) FROM employee;
  - **Explanation:** The COUNT(*) function counts all rows in the employee table to give the total number of employees.
- **Find the average salary of all employees.**
  - **Query:** SELECT AVG(salary) FROM employee;
  - **Explanation:** The AVG() function calculates the average value of the salary column across all employees.
- **Find the average salary of male and female employees.**
  - **Query:** SELECT gender, AVG(salary) FROM employee GROUP BY gender;

- **Explanation:** This query uses AVG(salary) to calculate the average salary, but the GROUP BY gender clause ensures that the calculation is performed separately for each unique value in the gender column (e.g., 'male' and 'female').

- **Find the total number of male and female employees and their average salary.**
  - **Query:** SELECT gender, COUNT(*) AS total_employees, AVG(salary) AS average_salary FROM employee GROUP BY gender;
  - **Explanation:** This query combines two aggregate functions: COUNT(*) to count the employees in each group and AVG(salary) to find their average salary. The GROUP BY gender clause organizes the data by gender, and column aliases are used to make the output headers more descriptive.
- **Find the name, post, and salary of the employee having the maximum salary (using a subquery).**
  - **Query:** SELECT name, post, salary FROM employee WHERE salary = (SELECT MAX(salary) FROM employee);
  - **Explanation:** This query first uses a subquery (SELECT MAX(salary) FROM employee) to find the single highest salary value in the table. The outer query then uses the WHERE clause to filter the employee table and return the details of the employee whose salary matches that maximum value.
- **Find the total amount the company must pay as salary per month.**
  - **Query:** SELECT SUM(salary) FROM employee;
  - **Explanation:** The SUM() function adds up all the values in the salary column to provide the total amount the company pays in salaries.

3. DEMONSTRATIONS

The base table:

```
MariaDB [ashishlabreport]> create table employee (eid integer primary key, name
 text, post text, salary integer, gender text, address text, contact text);
Query OK, 0 rows affected (0.040 sec)

MariaDB [ashishlabreport]> desc employee;
+---------+---------+------+-----+---------+-------+
| Field   | Type    | Null | Key | Default | Extra |
+---------+---------+------+-----+---------+-------+
| eid     | int(11) | NO   | PRI | NULL    |       |
| name    | text    | YES  |     | NULL    |       |
| post    | text    | YES  |     | NULL    |       |
| salary  | int(11) | YES  |     | NULL    |       |
| gender  | text    | YES  |     | NULL    |       |
| address | text    | YES  |     | NULL    |       |
| contact | text    | YES  |     | NULL    |       |
+---------+---------+------+-----+---------+-------+
7 rows in set (0.025 sec)
```

1. **Count the total number of employees in the 'employee' table.**

```
MariaDB [ashishlabreport]> select count(*) from employee;
+----------+
| count(*) |
+----------+
|       10 |
+----------+
1 row in set (0.001 sec)
```

2. **Find the average salary of all employees.**

```
MariaDB [ashishlabreport]> select avg(salary) from employee;
+-------------+
| avg(salary) |
+-------------+
|  54400.0000 |
+-------------+
1 row in set (0.001 sec)
```

3. **Find the average salary of male and female employees.**

```
MariaDB [ashishlabreport]> select gender, avg(salary) from employee group by ge
nder;
+--------+-------------+
| gender | avg(salary) |
+--------+-------------+
| female |  48250.0000 |
| male   |  58500.0000 |
+--------+-------------+
2 rows in set (0.020 sec)
```

4. **Find the total number of male and female employees and their average salary.**

```
MariaDB [ashishlabreport]> select gender, count(*), avg(salary) from employee group by gender;
+--------+----------+-------------+
| gender | count(*) | avg(salary) |
+--------+----------+-------------+
| female |        4 |  48250.0000 |
| male   |        6 |  58500.0000 |
+--------+----------+-------------+
2 rows in set (0.025 sec)
```

5. **Find the name post and salary of the employee having maximum salary (use subquery)**

```
MariaDB [ashishlabreport]> select name, post, salary from employee where salary
 = (select max(salary) from employee);
+---------------+---------+--------+
| name          | post    | salary |
+---------------+---------+--------+
| rajesh sharma | manager |  75000 |
+---------------+---------+--------+
1 row in set (0.001 sec)
```

6. **Find the total amount the company must pay as salary per month.**

```
MariaDB [ashishlabreport]> select sum(salary) from employee;
+-------------+
| sum(salary) |
+-------------+
|      544000 |
+-------------+
1 row in set (0.001 sec)
```

7. **How much does the company pay to all the females as salary?**

```
MariaDB [ashishlabreport]> select sum(salary) from employee where gender = 'female';
+-------------+
| sum(salary) |
+-------------+
|      193000 |
+-------------+
1 row in set (0.001 sec)
```

8. **Find the name of an employee having a salary less than average.**

```
MariaDB [ashishlabreport]> select name from employee where salary < (select avg(salary) from employee);
+-----------------+
| name            |
+-----------------+
| sita basnet     |
| isha thapa      |
| bikram tamang   |
| pujan magar     |
| aayusha ghimire |
+-----------------+
5 rows in set (0.001 sec)
```

4. RESULT AND DISCUSSION

The experiment demonstrated how the ALTER command can be used to update the schema of an existing table. We successfully added new columns, dropped unnecessary ones, modified data types, changed column names, and even renamed the table. The use of constraints such as primary keys was also shown. This illustrates that ALTER is a versatile command for schema evolution in MySQL.

5. CONCLUSION

The ALTER command is one of the most powerful tools in SQL for managing changes in table structures. It allows database designers to adapt to changing requirements without losing data. This experiment highlighted various operations like adding/removing columns, modifying properties, and renaming tables. Understanding and applying ALTER properly ensures efficient and flexible database management.

Experiment No: 9

TITLE: TO CREATE DATABASE SCHEMA AND RUN THE DIFFERENT UPDATE, DELETE AND ALTER QUERIES USING MYSQL.

1. OBJECTIVES

1.1 To understand the use of UPDATE, DELETE, and ALTER commands in MySQL.
1.2 To modify records and structure of a table using different SQL queries.
1.3 To analyze how update and delete operations affect the stored data.

2. THEORY

In MySQL, **Data Manipulation Language (DML)** and **Data Definition Language (DDL)** commands are used to manage both data and schema. The UPDATE command modifies existing records in a table, while the DELETE command removes specific records based on conditions. The ALTER command modifies the structure of an existing table by adding, deleting, or renaming columns and even renaming the table itself.

Data Manipulation Language (DML) Queries

1. Increase salary of all employees by 10%.

- **Query:** UPDATE employee SET salary = salary * 1.1;
- **Explanation:** The UPDATE statement modifies existing records in a table. By not including a WHERE clause, this query affects **all** rows in the employee table, multiplying each employee's salary by 1.1 to increase it by 10%.

2. List out different cities and update the postal code for each.

- **Query (Example for Pokhara):** UPDATE employee SET postal_code = 33600 WHERE city = 'pokhara';
- **Explanation:** This UPDATE query uses a WHERE clause to selectively apply changes. It will only update the postal_code column to 33600 for records where the city column is 'pokhara'. You would repeat this for other cities as needed.

3. Add a * sign after the post for employees with above-average salary.

- **Query:** UPDATE employee SET post = CONCAT(post, '*') WHERE salary > (SELECT AVG(salary) FROM employee);
- **Explanation:** This advanced UPDATE query uses a **subquery** (SELECT AVG(salary) FROM employee) to first calculate the average salary. The WHERE clause then filters for employees whose salary is greater than that average. The CONCAT() function is used to append a * to the end of their post string.

4. Delete records of employees from Pokhara and who are male.

- **Query:** DELETE FROM employee WHERE city = 'pokhara' AND gender = 'm';

- **Explanation:** The DELETE statement removes entire rows from a table. The WHERE clause is critical here, as it ensures that only records that meet **both** conditions (city is 'pokhara' AND gender is 'm') are permanently removed.

Data Definition Language (DDL) Queries

1. Add a new column postal_code after city.

- **Query:** ALTER TABLE employee ADD COLUMN postal_code INT AFTER city;
- **Explanation:** The ALTER TABLE command modifies a table's structure. ADD COLUMN creates a new column named postal_code with a data type of INT. The AFTER city clause specifies the new column's position, placing it immediately after the city column.

2. Change the name of the table to emp.

- **Query:** ALTER TABLE employee RENAME TO emp;
- **Explanation:** This ALTER TABLE command renames the table from employee to emp. All data and existing table properties remain the same.

3. Change the name of the column name to full_name.

- **Query:** ALTER TABLE emp CHANGE name full_name VARCHAR(30) NOT NULL;
- **Explanation:** The CHANGE clause in ALTER TABLE is used to rename a column. You must specify the old name (name), the new name (full_name), and its complete new definition (VARCHAR(30) NOT NULL), even if the data type and constraints are staying the same.

3. DEMONSTRATIONS

Table creation:

```
MariaDB [ashishlabreport]> desc employee;
+---------+-------------+------+-----+---------+-------+
| Field   | Type        | Null | Key | Default | Extra |
+---------+-------------+------+-----+---------+-------+
| eid     | int(11)     | NO   | PRI | NULL    |       |
| name    | varchar(50) | YES  |     | NULL    |       |
| post    | varchar(50) | YES  |     | NULL    |       |
| salary  | int(11)     | YES  |     | NULL    |       |
| gender  | varchar(10) | YES  |     | NULL    |       |
| city    | varchar(50) | YES  |     | NULL    |       |
| contact | varchar(15) | YES  |     | NULL    |       |
+---------+-------------+------+-----+---------+-------+
7 rows in set (0.024 sec)
```

- **Increase salary of all employee by 10%.**

Before:

```
MariaDB [ashishlabreport]> select eid, name, salary from employee;
+-----+-----------------+--------+
| eid | name            | salary |
+-----+-----------------+--------+
| 101 | rajesh sharma   |  82500 |
| 102 | sita basnet     |  49500 |
| 103 | narayan karki   |  60500 |
| 104 | isha thapa      |  52800 |
| 105 | sanjay rai      |  71500 |
| 106 | pratima dhakal  |  66000 |
| 107 | bikram tamang   |  46200 |
| 108 | pujan magar     |  57200 |
| 109 | aayusha ghimire |  44000 |
| 110 | sushil gurung   |  68200 |
+-----+-----------------+--------+
10 rows in set (0.001 sec)
```

After:

```
MariaDB [ashishlabreport]> update employee set salary = (salary * 1.1);
Query OK, 10 rows affected (0.006 sec)
Rows matched: 10  Changed: 10  Warnings: 0

MariaDB [ashishlabreport]> select eid, name, salary from employee;
+-----+-----------------+--------+
| eid | name            | salary |
+-----+-----------------+--------+
| 101 | rajesh sharma   |  90750 |
| 102 | sita basnet     |  54450 |
| 103 | narayan karki   |  66550 |
| 104 | isha thapa      |  58080 |
| 105 | sanjay rai      |  78650 |
| 106 | pratima dhakal  |  72600 |
| 107 | bikram tamang   |  50820 |
| 108 | pujan magar     |  62920 |
| 109 | aayusha ghimire |  48400 |
| 110 | sushil gurung   |  75020 |
+-----+-----------------+--------+
10 rows in set (0.001 sec)
```

- **Add a new column ‘postal_code’ after city.**

```
MariaDB [ashishlabreport]> alter table employee add column postal_code int after city;
Query OK, 0 rows affected (0.012 sec)
Records: 0  Duplicates: 0  Warnings: 0

MariaDB [ashishlabreport]> desc employee;
+-------------+-------------+------+-----+---------+-------+
| Field       | Type        | Null | Key | Default | Extra |
+-------------+-------------+------+-----+---------+-------+
| eid         | int(11)     | NO   | PRI | NULL    |       |
| name        | varchar(50) | YES  |     | NULL    |       |
| post        | varchar(50) | YES  |     | NULL    |       |
| salary      | int(11)     | YES  |     | NULL    |       |
| gender      | varchar(10) | YES  |     | NULL    |       |
| city        | varchar(50) | YES  |     | NULL    |       |
| postal_code | int(11)     | YES  |     | NULL    |       |
| contact     | varchar(15) | YES  |     | NULL    |       |
+-------------+-------------+------+-----+---------+-------+
8 rows in set (0.023 sec)
```

- **List out different cities in the table using select query and update the postal code for each city.**

```
MariaDB [ashishlabreport]> select city from employee;
+------------+
| city       |
+------------+
| kathmandu  |
| pokhara    |
| biratnagar |
| pokhara    |
| bhaktapur  |
| kathmandu  |
| pokhara    |
| birtamod   |
| kathmandu  |
| pokhara    |
+------------+
10 rows in set (0.001 sec)

MariaDB [ashishlabreport]> update employee set postal_code = 44600 where city = 'kathmandu';
Query OK, 3 rows affected (0.005 sec)
Rows matched: 3  Changed: 3  Warnings: 0

MariaDB [ashishlabreport]> update employee set postal_code = 33700 where city = 'pokhara';
Query OK, 4 rows affected (0.002 sec)
Rows matched: 4  Changed: 4  Warnings: 0

MariaDB [ashishlabreport]> update employee set postal_code = 56700 where city = 'biratnagar';
Query OK, 1 row affected (0.002 sec)
Rows matched: 1  Changed: 1  Warnings: 0

MariaDB [ashishlabreport]> update employee set postal_code = 44800 where city = 'bhaktapur';
Query OK, 1 row affected (0.002 sec)
Rows matched: 1  Changed: 1  Warnings: 0

MariaDB [ashishlabreport]> update employee set postal_code = 57100 where city = 'birtamod';
Query OK, 1 row affected (0.005 sec)
Rows matched: 1  Changed: 1  Warnings: 0
```

- **Add a * sign after the post of those employees who have salary more than average.**

```
MariaDB [ashishlabreport]> update employee set post = concat(post, '*') where salary > (select avg(salary)
from (select * from employee) as temp);
Query OK, 5 rows affected (0.007 sec)
Rows matched: 5  Changed: 5  Warnings: 0

MariaDB [ashishlabreport]> select eid , name, post from employee;
+-----+-----------------+-------------+
| eid | name            | post        |
+-----+-----------------+-------------+
| 101 | rajesh sharma   | manager*    |
| 102 | sita basnet     | developer   |
| 103 | narayan karki   | accountant* |
| 104 | isha thapa      | developer   |
| 105 | sanjay rai      | team lead*  |
| 106 | pratima dhakal  | hr manager* |
| 107 | bikram tamang   | developer   |
| 108 | pujan magar     | accountant  |
| 109 | aayusha ghimire | developer   |
| 110 | sushil gurung   | hr manager* |
+-----+-----------------+-------------+
10 rows in set (0.001 sec)
```

- **Delete the records of those employees who are from pokhara and are male.**

```
MariaDB [ashishlabreport]> delete from employee where city = 'pokhara' and gender = 'male';
Query OK, 2 rows affected (0.006 sec)

MariaDB [ashishlabreport]> select eid , name, city, gender from employee;
+-----+-----------------+------------+--------+
| eid | name            | city       | gender |
+-----+-----------------+------------+--------+
| 101 | rajesh sharma   | kathmandu  | male   |
| 102 | sita basnet     | pokhara    | female |
| 103 | narayan karki   | biratnagar | male   |
| 104 | isha thapa      | pokhara    | female |
| 105 | sanjay rai      | bhaktapur  | male   |
| 106 | pratima dhakal  | kathmandu  | female |
| 108 | pujan magar     | birtamod   | male   |
| 109 | aayusha ghimire | kathmandu  | female |
+-----+-----------------+------------+--------+
8 rows in set (0.001 sec)
```

- **Change the name of the table to ‘emp’.**

```
MariaDB [ashishlabreport]> alter table employee rename to emp;
Query OK, 0 rows affected (0.014 sec)

MariaDB [ashishlabreport]> show tables;
+---------------------------+
| Tables_in_ashishlabreport |
+---------------------------+
| emp                       |
+---------------------------+
1 row in set (0.001 sec)
```

- **Change the name of the column ‘name’ to ‘full_name’.**

```
MariaDB [ashishlabreport]> alter table emp change name full_name varchar(50)
Query OK, 0 rows affected (0.012 sec)
Records: 0  Duplicates: 0  Warnings: 0

MariaDB [ashishlabreport]> desc emp;
+-------------+-------------+------+-----+---------+-------+
| Field       | Type        | Null | Key | Default | Extra |
+-------------+-------------+------+-----+---------+-------+
| eid         | int(11)     | NO   | PRI | NULL    |       |
| full_name   | varchar(50) | YES  |     | NULL    |       |
| post        | varchar(50) | YES  |     | NULL    |       |
| salary      | int(11)     | YES  |     | NULL    |       |
| gender      | varchar(10) | YES  |     | NULL    |       |
| city        | varchar(50) | YES  |     | NULL    |       |
| postal_code | int(11)     | YES  |     | NULL    |       |
| contact     | varchar(15) | YES  |     | NULL    |       |
+-------------+-------------+------+-----+---------+-------+
8 rows in set (0.021 sec)
```

4. RESULT AND DISCUSSION

The experiment was carried out successfully. The UPDATE command increased the salary of all employees by 10%, and specific postal codes were assigned based on the city. A subquery was used to mark employees with above-average salary by appending an asterisk to their post. The DELETE command removed male employees from Pokhara. Using the ALTER command, a new column was added, the table name was changed to emp, and the column name was renamed to full_name. This experiment demonstrated both data-level and schema-level modifications effectively.

5. CONCLUSION

This experiment provided hands-on experience with the UPDATE, DELETE, and ALTER commands in MySQL. We learned how to increase salaries, assign postal codes, mark employees with high salaries, and remove specific records. Additionally, structural changes were applied by renaming the table and columns. These commands are powerful for maintaining and evolving database systems in real-world applications.

Experiment No: 10

TITLE: TO QUERY THE RELATIONAL DATABASE IN ORDER TO RETRIEVE THE REQUIRED INFORMATION BY USING THE JOIN OPERATIONS.

1. OBJECTIVES

1.1 To understand the use of the DROP command in MySQL.
1.2 To learn how to remove database objects like tables, databases, and constraints.
1.3 To analyze the effect of DROP on the schema and stored data.

2. THEORY

A **view** in MySQL is a **virtual table** based on the result of an SQL query. Unlike physical tables, views do not store data themselves; they display data stored in underlying tables. Views are especially useful for simplifying complex queries, enhancing data security by restricting access to sensitive columns, and providing customized representations of data.

Creating a View

- **Syntax:**
- CREATE VIEW view_name AS
- SELECT column1, column2, ...
- FROM table_name
- WHERE condition;
- **Explanation:**
    - CREATE VIEW view_name starts the command and gives the view a name.
    - AS is a required keyword that separates the view name from the query.
    - SELECT ... is the query that defines what data the view will contain.

- **Key Characteristics:**
    - Defined using the create view statement.
    - Acts like a table but does not store data physically.
    - Can be used in select queries as a normal table.
    - Supports abstraction by hiding complexity of queries.

Advantages:

- Simplifies query writing by encapsulating complex queries.
- Enhances security by allowing restricted access to specific columns.

- Provides logical independence between user queries and underlying schema.

Disadvantages:

- Performance overhead for very complex views.
- Cannot always perform insert, update, or delete on views depending on their definition.

3. DEMONSTRATIONS

Schema: employee(eid, name, post, salary, gender, city))

```
MariaDB [ashishlabreport]> create table employee (eid int primary key, name varchar(100),
 post varchar(50), salary decimal(10,2), gender varchar(10), city varchar(50));
Query OK, 0 rows affected (0.015 sec)

MariaDB [ashishlabreport]> desc employee;
+--------+---------------+------+-----+---------+-------+
| Field  | Type          | Null | Key | Default | Extra |
+--------+---------------+------+-----+---------+-------+
| eid    | int(11)       | NO   | PRI | NULL    |       |
| name   | varchar(100)  | YES  |     | NULL    |       |
| post   | varchar(50)   | YES  |     | NULL    |       |
| salary | decimal(10,2) | YES  |     | NULL    |       |
| gender | varchar(10)   | YES  |     | NULL    |       |
| city   | varchar(50)   | YES  |     | NULL    |       |
+--------+---------------+------+-----+---------+-------+
6 rows in set (0.021 sec)
```

Table data:

```
MariaDB [ashishlabreport]> select * from employee;
+-----+---------------+-----------+----------+--------+-----------+
| eid | name          | post      | salary   | gender | city      |
+-----+---------------+-----------+----------+--------+-----------+
|   1 | ram sharma    | manager   | 60000.00 | male   | kathmandu |
|   2 | sita rai      | developer | 45000.00 | female | lalitpur  |
|   3 | hari karki    | developer | 42000.00 | male   | pokhara   |
|   4 | sophia gurung | analyst   | 38000.00 | female | pokhara   |
|   5 | suman khadka  | hr        | 50000.00 | male   | butwal    |
+-----+---------------+-----------+----------+--------+-----------+
5 rows in set (0.001 sec)
```

Create view #1 (simple “basic details” view):

```
MariaDB [ashishlabreport]> create view employee_basic as select eid, name, post from employee;
Query OK, 0 rows affected (0.008 sec)

MariaDB [ashishlabreport]> select * from employee_basic;
+-----+---------------+-----------+
| eid | name          | post      |
+-----+---------------+-----------+
|   1 | ram sharma    | manager   |
|   2 | sita rai      | developer |
|   3 | hari karki    | developer |
|   4 | sophia gurung | analyst   |
|   5 | suman khadka  | hr        |
+-----+---------------+-----------+
5 rows in set (0.002 sec)
```

Create view #2 (high salary employees) :

```
MariaDB [ashishlabreport]> create view high_salary as select name, post, salary from employee where salary > 45000;
Query OK, 0 rows affected (0.009 sec)

MariaDB [ashishlabreport]>
MariaDB [ashishlabreport]> select * from high_salary;
+--------------+---------+----------+
| name         | post    | salary   |
+--------------+---------+----------+
| ram sharma   | manager | 60000.00 |
| suman khadka | hr      | 50000.00 |
+--------------+---------+----------+
2 rows in set (0.002 sec)
```

Create view #3 (female employees):

```
MariaDB [ashishlabreport]> create view female_employees as select name,
 city, salary from employee where gender='female';
Query OK, 0 rows affected (0.008 sec)

MariaDB [ashishlabreport]> select * from female_employees;
+---------------+----------+----------+
| name          | city     | salary   |
+---------------+----------+----------+
| sita rai      | lalitpur | 45000.00 |
| sophia gurung | pokhara  | 38000.00 |
+---------------+----------+----------+
2 rows in set (0.002 sec)
```

list all views in the db:

```
MariaDB [ashishlabreport]> show full tables where table_type='VIEW';
+----------------------------+------------+
| Tables_in_ashishlabreport | Table_type |
+----------------------------+------------+
| employee_basic             | VIEW       |
| female_employees           | VIEW       |
| high_salary                | VIEW       |
+----------------------------+------------+
3 rows in set (0.001 sec)
```

4. RESULT AND DISCUSSION

The experiment was successfully performed by creating three views in the employee schema. The first view, employee_basic, displayed only the essential details of employees such as ID, name, and post. The second view, high_salary, showed the list of employees earning more than 45,000. The third view, female_employees, listed only the female employees with their city and salary. These views simplified queries, restricted access to certain information, and demonstrated the usefulness of views in real-world database applications.

6. CONCLUSION

This experiment demonstrated how to create and use views in MySQL. Views provide a way to simplify queries, enhance security by limiting column access, and offer an abstraction layer between users and the actual database schema. By creating and querying views, the importance of views in database management was clearly understood.